

FINAL TECHNICAL REPORT(Till now-just planner)

AI Job Tracker Web Application

1. SYSTEM OVERVIEW

This project is a **full-stack job tracking web application** that:

- ✓ Automatically stores job data
 - ✓ Displays jobs in dashboard UI
 - ✓ Allows real-time status updates
 - ✓ Syncs data with backend automation
-

2. SYSTEM ARCHITECTURE

Frontend (React + Vite)

↓

Fetch API (HTTP Requests)

↓

Webhook API (n8n)

↓

Google Sheets (Database)

3. FRONTEND STACK

Used technologies:

- React
- Vite
- JavaScript
- HTML
- CSS
- lucide-react

Responsibilities:

- ✓ Render job dashboard
 - ✓ Search & filter jobs
 - ✓ Modal UI for editing
 - ✓ Optimistic UI updates
 - ✓ API communication
-

4. BACKEND STACK (AUTOMATION)

Used:

- n8n

Responsibilities:

- ✓ Acts as backend server
 - ✓ Exposes Webhook API
 - ✓ Processes incoming data
 - ✓ Updates Google Sheets
 - ✓ Handles automation workflows
-

5. DATABASE

Used:

- Google Sheets

Structure:

Column	Purpose
--------	---------

Company	Company name
---------	--------------

Role	Job role
------	----------

Status	Interview / Offer / Rejected
--------	------------------------------

Notes	Description
-------	-------------

Column	Purpose
--------	---------

Date	Event date
------	------------

Responsibilities:

- ✓ Persistent data storage
- ✓ Simple database alternative
- ✓ Easily accessible via API

6. APIs USED (IMPORTANT)

◆ 6.1 Google Sheets Data API (via gviz)

GET <https://docs.google.com/spreadsheets/d/{sheetId}/gviz/tq?tqx=out:json>

Purpose:

- ✓ Fetch all job data
- ✓ Used in React (fetchJobs)
- ✓ Returns structured JSON

◆ 6.2 Webhook API (n8n)

POST <http://localhost:5678/webhook/update-job>

Request Body:

```
{  
  "row_number": 2,  
  "status": "Offer"  
}
```

Purpose:

- ✓ Receive data from frontend
- ✓ Trigger workflow
- ✓ Update specific row in sheet

◆ 6.3 Fetch API

Used:

- Fetch API

Purpose:

✓ Send POST request to webhook

✓ Fetch data from Google Sheets

✓ Handle async operations

◆ 6.4 Testing API Tool

Used:

- Postman

Purpose:

✓ Test webhook independently

✓ Debug request/response flow

7. DATA FLOW

● READ FLOW

React → Fetch API → Google Sheets API → UI Render

● UPDATE FLOW

User क्लिक →

React →

POST (Webhook API) →

n8n →

Google Sheets →

8. KEY FEATURES IMPLEMENTED

- ✓ Job dashboard UI
 - ✓ Search functionality
 - ✓ Filter by status
 - ✓ Modal-based editing
 - ✓ Real-time optimistic updates
 - ✓ Backend integration via webhook
 - ✓ Persistent storage
-

9. ADVANCED CONCEPTS USED

- ✓ Full-stack integration
 - ✓ REST API communication
 - ✓ Webhook architecture
 - ✓ Optimistic UI update
 - ✓ Asynchronous programming (async/await)
 - ✓ State management (React Hooks)
-

10. PROJECT TYPE

- ✓ Full Stack Web Application
 - ✓ No-Code + Code Hybrid System
 - ✓ Automation-driven SaaS prototype
-

11. INDUSTRY COMPARISON

Your Stack	Industry Equivalent
------------	---------------------

React	Frontend (same)
-------	-----------------

n8n	Node.js / Express
-----	-------------------

Google Sheets	MongoDB / MySQL
---------------	-----------------

Webhook API	REST API
-------------	----------

12. FUTURE IMPROVEMENTS

- ✓ Replace Sheets → MongoDB
 - ✓ Replace n8n → Express backend
 - ✓ Add authentication system
 - ✓ Deploy on Vercel
 - ✓ Real-time updates without polling
-

13. FINAL SUMMARY (FOR INTERVIEW)

 You can say:

"I built a full-stack job tracking system using React for frontend, n8n as backend via webhook APIs, and Google Sheets as database. The app supports real-time status updates using REST API communication and implements optimistic UI for better user experience."

FINAL VERDICT

- ✓ Not a basic project
- ✓ Full-stack system
- ✓ API-driven architecture
- ✓ Automation integrated